Competency 3: Investigates how instructions and data are represented in computers and exploit them in arithmetic and logic operations

Competency level 3.1: Analyses how numbers are represented in computers

Time: 10 periods

Learning outcomes

- Describes that instructions and data are represented using two states in computers
- Explains the need of different number systems
- Converts decimal integers into binary, octal and hexadecimal numbers and vice versa
- Converts binary numbers to octal and hexadecimal, and vice versa
- Converts octal numbers to binary and hexadecimal, and vice versa
- Describes how different types of decimal numbers are stored in computers
- Explains that the Most Significant Bit (MSB) is used to indicate the sign when converting the given binary value to one's complement
- Converts a given binary value to two's complement
- Explains the usage of one's complement and two's complement

Contents:

- Need for instruction and data representation in digital devices
- Methods of instruction and data representation in computers
    - Representation of data in two states (0,1)
- Number systems used in computing
    - Binary, octal, hexadecimal
    - Conversion between number systems
- Representation of decimal numbers (signed and unsigned)
    - Signed integer representations
    - Signed magnitude
    - One's complement
    - Two's Complement

Concepts and terms to be highlighted:

- A set of rules have to be followed in converting a number from one system to another.
- A standard procedure is available to present the digits of the converted number
- The fixed point and floating point formats for decimal arithmetic
- The representation of negative numbers using sign-magnitude, one's complement and two's complement methods.

Guidance for lesson plans:

Let‘s inquire into methods of representing computer related numerical data.

Pay attention to the following conversions.

- Decimal Numbers convert as Binary, Octal and Hexadecimal
- Binary Numbers convert as Octal and Hexadecimal
- Octal Numbers convert as Decimal, Binary and Hexadecimal
- Hexadecimal Numbers convert as decimal Binary, and Octal

Guidance for assessments and evaluations:

- Give enough questions to get familiar with the number conversions.

Quality inputs:

- Presentations, Internet facility

Reading Material

Digital devices represent everything as numbers.
– Numbers, characters, symbols, images, audio and video and all the multimedia items and instructions are represented by numbers in digital devices.
All modern digital devices use binary numbers (base 2) instead of decimal (base 10)
Smallest unit in digital device is bit.
– A bit is a "binary digit", that is, a number that is either 0 or 1
– Computers ultimately represent and process everything as bits
Groups of bits represent larger things
– Numbers, letters, words, names, pictures, sounds, instructions,..
– The interpretation of a group of bits depends on their context

Humans need different number systems to interact with computers. Humans can't remember large set of ones and zeros ("1" and "0"). Binary files are difficult for humans to read and edit. Binary files can get confusing when transferring between computers with different architectures. By using octal and hexadecimal numbers humans can easily read.

Decimal Numbers

- The number system is extremely close to our day to day life.
- It is made up of the digits of 0,1,2,3,4,5,6,7,8,9
- It is called the decimal number system because ten digits available in this number system
- It can be believed that it has become a popular number system because humans have ten fingers in their hand.
- However, a large number can be represented using these ten digits.
- The decimal point is used to separate a fractional part of a number
- Plus "+" sign used to show positive values and negative "-" sign is used to show negative values.
  E.g. :- +10.235
  -25.321

Integers

- Integers are a number set.
- Natural numbers (0, 1, 2, 3...) and (-1,-2,-3,…) are integers.
- Neither decimal number nor fractions are integers.
  E.g. :- 0.75, 8.5

Decimal number system

- This number system has 10 digits of 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- When the value of a particular number exceeds the largest number 9 in that number set, the multiples of 10 of the number of values are transferred to the next (left) place value. Every place value is multiplied by ten to get the next place value.

  E.g. :- 3456 $= 3 \times 10^3 + 4 \times 10^2 + 5 \times 10^1 + 6 \times 10^0$
  $= 3000 + 400 + 50 + 6$
  $= 3456$

  The place values in decimal number are multiple values of 10. Therefore the base value of the decimal number system is 10.

Binary number system

- The binary number system has two digits which can be represent two states.
- These two states are, represented by digits “0” and “1”.
- Therefore a number system with the two digits can be used here.
- There are multiplications of 2 in place values of the binary number system. They are as follows.

2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	
16	8	4	2	1	1/2	1/4	1/8	Place values

- Therefore, the base value of the binary number system is 2.
- As the computer works on electricity and is an electronic device, its functions are controlled by two states.
- These two states are, where the power is ON and OFF (As two different levels of voltage)
- The every place value is multiply by 0 or 1 (digits of binary number system) to get the value of binary number.

E.g. :- $11010_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$
$= 1x16+1x8+0x4+1x2+0x1$
$= 26_{10}$

Therefore, $11010_2 = 26_{10}$

One location (one digits) is call a bit. There are 5 bits in the above number.

Octal number system

- The base value of the octal number system is 8.
- The digits are 0, 1, 2, 3, 4, 5, 6 and 7.
- The place values are as follows.

8^2	8^1	8^0	8^{-1}	8^{-2}	
64	8	1	1/8	1/64	place values

E.g. :- $673_8 = 6 \times 8^2 + 7 \times 8^1 + 3 \times 8^0$
$= 6x64+7x8+3x1$
$= 443_{10}$

Therefore, $673_8 = 443_{10}$

Hexadecimal number system

- The base value of hexadecimal number system is16.
- There are 16 digits in the hexadecimal number system. Digits over value 9 need two digits. Therefore, A, B, C, D, E, F also used as remaining digits. All digits are as follows.
  Digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- The minimum value is 0 and maximum value is F ($=15_{10}$).
- The values represented by the digits are as follows

Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

E.g. :- $BC12_{16} = B(11) \times 16^3 + C(12) \times 16^2 + 1 \times 16^1 + 2 \times 16^0$
$= 11\times16^3 + 12\times16^2 + 1\times16^1 + 2\times16^0$
$= 11x4096 + 12 x256+ 1x16+2x1$
$= 45056 + 3072 + 16 + 2$
$= 48146$

Therefore, $BC12_{16} = 48146_{10}$

The Decimal number 765.43 can be tabulated as follows.

	100	10	1	1/10	1/100	
Place Value	10^2	10^1	10^0	10^{-1}	10^{-2}	
Numbers	7	6	5	4	3	765.43
Value	700	60	5	0.4	0.03	

Most Significant Digit (MSD) and Least Significant Digit (LSD)

MSD - The Digit that contain the most positional value in a number.
LSD - The Digit that contains the least positional value in a number.

Number	**MSD**	**LSD**
2975.0	2	5
56.034	5	4

E.g.:-

0.03145	3	5
0031.0060	3	6

Conversions between number systems

1. Conversion of a decimal numbers into a binary numbers
   - Divide the given decimal number by 2
   - Write the integer answer (quotient) under the long division symbol.
   - Write the remainder (0 or 1) to the right of the dividend.
   - Continue downwards, dividing each new quotient by 2 and writing the remainders to the right of each dividend. Stop when quotient is 0.
   - Starting from bottom, write the sequence of 1's and 0's upwards to the top.

E.g.:- Converting number 12_{10} to a binary number.

First, divide this number by 2 writing the remainders.

```
2 | 12     Remainder
2 | 6      0
 2 |  3    0
2 | 1      1
     0     1
```

Quotient

$12_{10} = 1100_2$

2. Conversion of a decimal numbers into a Octal numbers
   - Divide the given decimal number by 8
   - Write the integer answer (quotient) under the long division symbol.
   - Write the remainder (0 to 7) to the right of the dividend.
   - Continue downwards, dividing each new quotient by 8 and writing the remainders to the right of each dividend. Stop when quotient is 0.
   - Starting from bottom, write the sequence of remainders upwards to the top.

E.g.:- Converting number 245_{10} into an octal number.

```
8 | 245    Remainder
8 | 30     5
 8 |  3    6
     0     3
```

Quotient

$245_{10} = 365_8$

3. Conversion of a decimal numbers into a Hexadecimal numbers
   - Divide the given decimal number by 16. Treat the division as an integer division.
   - Write down the remainder (in hexadecimal, i.e. if the remainder is 12, write down "C").
   - Repeat step a and b until the result is 0.

- The hexadecimal value is the digit sequence of the remainders from the bottom to top.

E.g.:- Converting number 3240_{10} into a hexadecimal number.

```
16 |3240  Remainder
16 |  202  8
 16 |   12 10 (A)
        0 12 (C)
```

Quotient

$3240_{10} = CA8_{16}$

Converting fractions to Binary

- Multiply the given decimal fraction by 2.
- Multiply by 2 until the decimal part becomes 0.
- Write the values in front of decimal point from beginning to end.

E.g.:- convert 0.3125_{10} to binary

	0.3125	x2
0	.625	x2
1	.25	x2
0	.50	x2
1	.00	

$0.3125_{10} = 0.0101_2$

Converting fractions to Octal

- Multiply the given decimal fraction by 8.
- Multiply the decimal by 8 until it becomes 0.
- Write from the beginning to end, the values in front of the decimal point.

E.g. :- convert 0.3125_{10} to binary

0	0.3125	x8
2	.50	x8
4	.0	x8

$0.3125_{10} = 0.24_8$

Converting Binary Numbers to Decimal Numbers

E.g.:- Converting number 1101_2 to a decimal number.

1	1	0	1_2	
2^3	2^2	2^1	2^0	← Place values

$1101_2 = (1x2^3) + (1x2^2) + (0x2^1) + (0x2^0)$

$= (1x8) + (1x4) + (0x2) + (1x1)$

$= 8 + 4 + 0 + 1$

$1101_2 = 13_{10}$

Converting Octal Numbers to Decimal Numbers

E.g.:- Converting number 1260_8 to a decimal number.

1	2	6	0_8	
8^3	8^2	8^1	8^0 ←	Place values

$1260_8 = (1x8^3) + (2x8^2) + (6x8^1) + (0x8^0)$

$= (1x512) + (2x64) + (6x8) + (0x1)$

$= 512 + 128 + 48 + 0$

$1260_8 = 688_{10}$

Converting hexadecimal Numbers to Decimal Numbers

E.g.:- Converting number $A0B1_{16}$ to a decimal number.

A(10)	0	B(11)	1_{16}	
16^3	16^2	16^1	16^0 ←	Place values

$A0B1_{16} = (10x16^3) + (0x16^2) + (11x16^1) + (1x16^0)$

$= (10x4096) + (0x256) + (11x16) + (1x1)$

$= 40960 + 0 + 176 + 1$

$A0B1_{16} = 41137_{10}$

Converting Binary Numbers to Octal Numbers

Group all the 1's and 0's in the binary number in sets of three, starting from the far right. Add zeros to the left of the last digit if you don't have enough digits to make a set of three. Next assuming all groups a different 3 digit binary number multiply it by place values and get the sums separately. Put everything back together to get the octal number.

E.g. :- 10011011_2

Grouping: $10,011,011_2$

Adding Zeros for Groups of Three: $010, 011, 011_2$

$=(0x2^2+ 1x2^1 + 0x2^0), (0x2^2+ 1x2^1 + 1x2^0), (0x2^2+ 1x2^1 + 1x2^0)$

$=(0+2+0), (0+2+1), (0+2+1)$

$=2, 3, 3$

$=233_8$

Converting Binary Numbers to hexadecimal Numbers

Cut your string of binary numbers into groups of four, starting from the right.

Add extra zeros to the front of the first number if it is not four digits.

Convert one 4-digit group at a time and change to hexadecimal digits. Put everything back together to get the hexadecimal number

E.g.:- Convert 11101100101001_2 to hexadecimal

Grouping: 11,1011,0010,1001

= 0011,1011,0010,1001

$=(0x2^3 +0x2^2 +1x2^1 +1x2^0), (1x2^3 +0x2^2 +1x2^1 +1x2^0), (0x2^3 + 0x2^2 +1x2^1 +0x2^0),$

$(1x2^3 + 0x2^2 + 0x2^1 + 1x2^0)$
= (0+0+2+1), (8+0+2+1), (0+0+2+0), (8+0+0+1)
= 3, 11, 2, 9
= $3B29_{16}$
Therefore $11101100101001_2 = 3B29_{16}$

Converting Octal Numbers to Binary Numbers

Octal Number (Digit)	Binary Number
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Write the equivalent three binary digits groups for each octal digit. Remove the zeros from left which has no values. Put all together to get the binary equivalent number.

137_8 = 001,011,111
= 001011111_2
= 1011111_2

Converting hexadecimal Numbers to Binary Numbers

Write the equivalent four binary digits groups for each hexadecimal digit.
Remove the zeros from left which has no values.
Put all together to get the binary equivalent number.

$1A90_{16}$ = 0001,1010,1001,0000
= 00011010110010000_2
= 11010110010000_2

For negative numbers we need an alternative interpretation of bit values.
Three interpretations have been used:

Sign-Magnitude.

The sign and magnitude method is commonly an 8 bit system that uses the most significant bit (MSB) to indicate a positive or a negative value. By convention, a ‘0’ in this position indicates that the number given by the remaining 7 bits is positive, and a most significant bit of ‘1’ indicates that the number is negative. This interpretation makes it possible to create a value of negative zero.

E.g. :- $+45_{10}$ in signed binary is 00101101_2
- 45_{10} in signed binary is 10101101_2

One's Complement and Two's Complement

The functionality of subtraction using one's complement and two's complement is described using the below formula.

$$X - Y = X + 2^n - Y - 2^n$$
$$= X + \underbrace{2^n - 1 - Y}_{\text{One's Complement}} + 1 - 2^n$$
$$= X + (\underbrace{(2^n - 1 - Y)}_{\text{Two's Complement}} + 1) - 2^n$$

One's Complement of Y $= 2^n - 1 - Y$

Two's Complement of Y $= 2^n - 1 - Y + 1$

E.g. :- X=10, Y=5

$X - Y = X + 2^n - 1 - Y + 1 - 2^n$ $10_{10} - 5_{10} = 10_{10} + 2^8 - 1 - 5_{10} + 1 - 2^8$ $= 10_{10}$+1s Complement of $5_{10} + 1 - 2^8$ $= 00001010_2$ + 1's Complement of 00000101_2+ $1 - 2^8$	$10_{10} = 00001010_2$ $5_{10} = 00000101_2$ $2^8 = 100000000_2$ $2^8 - 1 = 100000000_2 - 00000001_2 = 11111111_2$
1's complement of $5 = 2^n - 1 - Y$ $= 2^8 - 1 - 00000101_2$ $= 11111111_2 - 00000101_2$	11111111_2- $\underline{00000101_2}$ 11111010_2 (No deduction is required. Here, 1s become 0s and 0s become 1s in 00000101_2)
1's complement of $5 = 11111010_2$	
2's complement of Y $= 2^n - 1 - Y + 1$ 2's complement of 5 $= 11111010_2 + 00000001_2$ $= 11111011_2$	11111010_2+ $\underline{00000001_2}$ 11111011_2
$10_{10} - 5_{10} = 00001010_2 + 11111011_2 - 2^8$	$X - Y = X + (2^n - 1 - Y + 1) - 2^n$ $= X +$ Two's complement of $Y - 2^n$
$= 100000101_2 - 2^8$	00001010_2+ $\underline{11111011_2}$ 100000101_2
$= 100000101_2 - 100000000_2$	100000101- $\underline{100000000}$ 00000101_2
$= 00000101_2$ $(=5_{10})$	(There is no deduction. Just only remove the first digit)

One's Complement

In one's complement, positive numbers are represented as usual in regular binary. However, negative numbers are represented differently. To negate a number, replace all zeros with ones, and ones with zeros - flip the bits. Thus, 12 would be 00001100, and -12 would be 11110011. As in signed magnitude, the leftmost bit (most significant bit-MSB) indicates the sign (1 is negative, 0 is positive). To compute the value of a negative number, flip the bits and translate as before.

When representing positive and negative numbers in 8-bit ones complement binary form, the positive numbers are the same as in signed binary notation.

E.g.: -120 is represented in one's complement form as 10000111_2 and

-60 is represented in one's complement form as 11000011_2

The ones complement system still has two ways of writing 0_{10} ($00000000_2 = +0_{10}$ and $11111111_2 = -0_{10}$);

Two's Complement.

A single set of bits is used. To form a negative number, start with a positive number, complement each bit and add one. This interpretation includes one more negative value than positive values (to accommodate zero).

E.g. :- -5 is represented in two's complement form as 11111011_2 and $+5_{10}$ as 00000101_2.

$+5= 00000101_2$, 1^s complement of 00000101_2 is 11111010_2

2^s complement of $00000101_2 = 11111010_2+1=11111011_2$

-5_{10} is represented in two's complement as 11111011_2 (In hexadecimal FB_{16})

	Usage
Sign Magnitude	Used only when we do not add or subtract the data. They are used in analog to digital conversions. They have limited use as they require complicated arithmetic circuits.
One's Complement	Simpler design in hardware due to simpler concept.
Two's Complement	Makes it possible to build low-cost, high-speed hardware to perform arithmetic operations.

Fixed point numbers

- In calculations involving fixed point numbers that have a fixed number of digits after the decimal point.

E.g. :- 763.2135 (The decimal point is located in the same position in each number)
179.4821
942.6956

Floating point

- The floating point number is used to represent
  - – Numbers with fractions, e.g., 3.1416
  - – Very small numbers, e.g., 0.000000001
  - – Very large numbers, e.g., 3.15576×10^9

e.g.:- -10.625 is represented in single precision floating point as below.
First convert it to binary 1010.101_2 (the whole and the fractional part separately)
Put it in standard form 1.010101×2^3
Bias the exponent by using excess- k method ($2^{k-1}-1$)
K for exponent 8 = $2^{8-1}-1=128-1=127$
Biased value for exponent = 3+127 = 130_{10}
exponent part in binary is 10000010_2

1	10000010	01010100000000000000000

– Sign, exponent, Mantissa: $(-1)^{sign} \times$ Mantissa $\times 2^{exponent}$
– More bits for Mantissa give more accuracy
– More bits for exponent increases range

• IEEE 754 floating point standard:
  – Single precision: 8 bit exponent, 23 bit Mantissa
  – Double precision: 11 bit exponent, 52 bit Mantissa

	Advantage	**Disadvantage**
Fixed Point Representation	Performance good. No need to rely on additional hardware or software logic.	Limited range of values can represent.
Floating point representation	Greater range of numbers is represented. Varying degrees of precision.	More storage space needed. Slower processing times. Lack of precision.

Competency level 3.2: Analyses how characters are represented in computers

Time: 04 periods

Learning outcomes

- Lists the methods of character representation in computer
- Converts given symbols into a representation scheme
- Describes the advantage and disadvantage of different data representation schemes

Contents:

- Methods of character representation
    - BCD
    - EBCDIC
    - ASCII
    - Unicode

Concepts and terms to be highlighted:

- Character representation on computers
- The advantages of using BCD
- Comparisons between EBCDIC and ASCII
- The advantages of using Unicode.

Guidance for lesson plans:

- Discuss the need for characters to be represented on computers
- Discuss the benefits of using binary coded decimal.
- Show the ASCII and EBCDIC tables and compare it.
- Discuss the necessity of Unicode.

Guidance for assessments and evaluations:

Give assignments to find out the relevant representation of characters.

Quality inputs:

- Presentations, Internet facility, ASCII Charts

Reading Material

Characters are represented on computers using several standard methods such as ASCII, BCD,
EBCDIC and UNICODE

BCD – (Binary Coded Decimal) CODE – This is a 4 bit code used for coding numeric values (0-9) only. 2^4=16 the remaining 6 (i.e. 1010, 1011, 1100, 1101, 1110, 1111) are invalid combinations.
E.g.:- $1000111_2 = 0100\ 0111_{BCD} = 47_{10}$

BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
Decimal	0	1	2	3	4	5	6	7	8	9

ASCII – ASCII (American Standard Codes for Information Interchange) normally uses 8 bits (1 byte) to store each character. However, the 8th bit is used as a check digit, meaning that only 7 bits are available to store each character. This gives ASCII the ability to store a total of 2^7 = 128 different values . The 7 bit ASCII code was originally proposed by the American National Standard Institute (ANSI). (IBM personal computers use ASCII).

EBCDIC (Extended Binary Coded Decimal Interchange Code) – The 8 bit EBCDIC is used primarily by large IBM mainframe computers and compatible equipment. It uses 256 different characters

UNICODE – The 16 bit code provides unique code points to characters in many of the world's languages including Sinhala and Tamil. One of the promising proposals is named Unicode.

	Advantage	**Disadvantage**
BCD	• Easy to encode and decode decimals into BCD and vice versa. • Simple to implement a hardware algorithm for the BCD converter. • It is very useful in digital systems whenever decimal information is given either as inputs or displayed as outputs. • Digital voltmeters, frequency converters and digital clocks all use BCD as they display output information in decimal.	• Not space efficient. • Difficult to represent the BCD form in high speed digital computers in arithmetic operations, especially when the size and capacity of their internal registers are restricted or limited. •Require a complex design of Arithmetic and logic Unit (ALU) than the straight Binary number system. •The speed of the arithmetic operations slow due to the complete hardware circuitry involved.
ASCII	• Uses a linear ordering of letters. • Different versions are mostly compatible. • compatible with modern encodings	• Not Standardized. • Not represent world languages.
EBCDIC	• uses 8 bits while ASCII uses 7 before it was extended.	• Does not use a linear ordering of letters.

	• Contained more characters than ASCII.	• Different versions are mostly not compatible. • Not compatible with modern encodings
UNICODE	• Standardized. • Represents most written languages in the world • ASCII has its equivalent within Unicode.	• Need twice memory to store ASCII characters.

Competency level 3.3: Uses basic arithmetic and logic operations on binary numbers

Time: 04 periods

Learning outcomes

- Works out additions (multiple numbers with or without carryovers) – in binary numbers
- Works out subtraction (with or without borrowing) –in binary numbers
- Works out NOT, AND, OR, XOR bitwise operations

Contents:

- Binary arithmetic operations - (integers only)
    - Addition, subtraction
- Logical operations
    - Bitwise logical operations

Concepts and terms to be highlighted:

- Binary number addition with/ without carry overs
- Binary number subtractions
- Bitwise logical operations using NOT, AND, OR, XOR operations.

Guidance for lesson plans:

- Add two binary numbers by writing one below another according to the place values.
- Subtract binary numbers by writing the small number below the big number.
- For the given binary number do the bitwise logical operations

Guidance for assessments and evaluations:
Give enough questions to get familiar with binary addition subtraction and bitwise operations.

Quality inputs:

- presentations

Reading Material

Addition of binary numbers

$101100_2 + 1100_2 =$

$$\begin{array}{l} 1\,0\,1\,1\,0\,0_2 \\ \underline{\quad 1\,1\,0\,0_2} \\ 1\,1\,1\,0\,0\,0_2 \end{array}$$

$101100_2 + 1100_2 = 111000_2$

Subtraction of binary numbers

$0010\ 110_2 - 001100_2$

$1\ 0\ 1\ 1\ 0\ 0_2$

$\underline{\quad 1\ 1\ 0\ 1_2}$

$0\ 1\ 1\ 1\ 1\ 1_2$

$101100_2 - 1101_2 = 011111_2$

Bitwise operations

1. NOT operation

   For unsigned integers, the bitwise complement of a number is the "mirror reflection" of the number across the half-way point of the unsigned integers range. One use is to invert a grayscale image where each pixel is stored as an unsigned integer. It uses the below bit operations.

A	**NOT A**
0	1
1	0

   E.g. :- **NOT** 0111_2 (7_{10}) = 1000_2 (8_{10})

2. Bitwise AND operation

   This is often called bit masking. (By analogy, the use of masking tape covers, or masks, portions that should not be altered or portions that is not of interest. In this case, the 0 values mask the bits that are not of interest.)

   It uses the below bit operation

A	**B**	**A AND B**
0	0	0
0	1	0
1	0	0
1	1	1

   E.g. :- 0101_2 (5_{10}) **AND** 0011_2 (3_{10})

   $0\ 1\ 0\ 1_2$

   $\underline{0\ 0\ 1\ 1_2}$

   $0\ 0\ 0\ 1_2$ (1_{10})

   Therefore 0101_2 **AND** 0011_2 is 0001_2

3. Bitwise OR operation

   A bitwise OR takes two bit patterns of equal length and performs the logical inclusive OR operation on each pair of corresponding bits. The result in each position is 0 if both bits are 0, while otherwise the result is 1. It uses the below bit operation

A	**B**	**A OR B**
0	0	0
0	1	1
1	0	1
1	1	1

E.g. :- 0101_2 (5_{10}) **OR** 0011_2 (3_{10})

$$\begin{array}{r} 0\ 1\ 0\ 1_2 \\ \underline{0\ 0\ 1\ 1_2} \\ 0\ 1\ 1\ 1_2\ (7_{10}) \end{array}$$

Therefore 0101_2 OR 0011_2 is 0111_2

4. Bitwise XOR operation

   The bitwise XOR may be used to invert selected bits in a register (also called toggle or flip). Any bit may be toggled by XOR it with 1

   It uses the below bit operations

A	**B**	**A XOR B**
0	0	0
0	1	1
1	0	1
1	1	0

E.g. :- 0010_2 (2_{10}) **XOR** 1010_2 (10_{10})

$$\begin{array}{l} 1\ 0\ 1\ 0_2 \\ \underline{0\ 0\ 1\ 0_2} \\ = 1\ 0\ 0\ 0_2\ (8_{10}) \end{array}$$

Therefore 0010_2 **XOR** 1010_2 is 1000_2